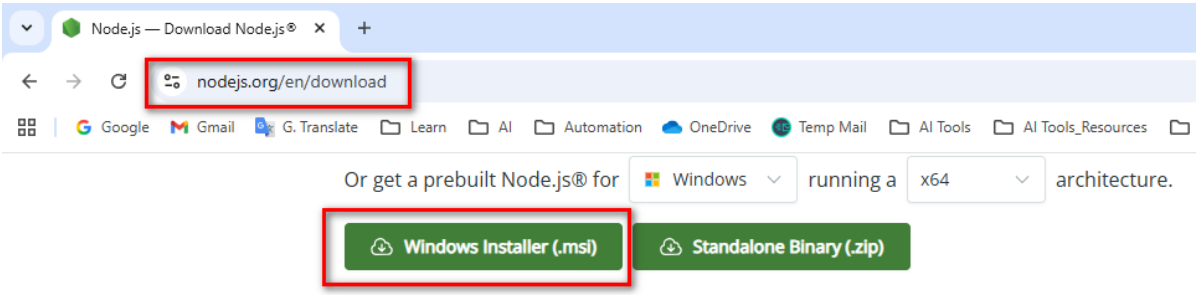
# Realtime e-commerce with MongoDB

**Technical stack and how each is used**

- **HTML5:** Semantic layout for navbar, header, product grid, modals.
- **CSS3:** Custom styles to refine Bootstrap components.
- **Bootstrap:** Responsive grid, cards, navbar, buttons via CDN.
- **NodeJS:** Runtime for server-side JavaScript and package scripts.
- **jQuery:** AJAX and DOM updates; listens to realtime events.
- **ExpressJS:** REST API and static file serving.
- **MongoDB:** Database for products and orders via Mongoose.
- **Socket.IO:** Realtime broadcasting of product updates and new orders.

## Download & Install NodeJS in your system




C:\WINDOWS\system32>node -v
**v24.11.1**

## Install and configure local MongoDB

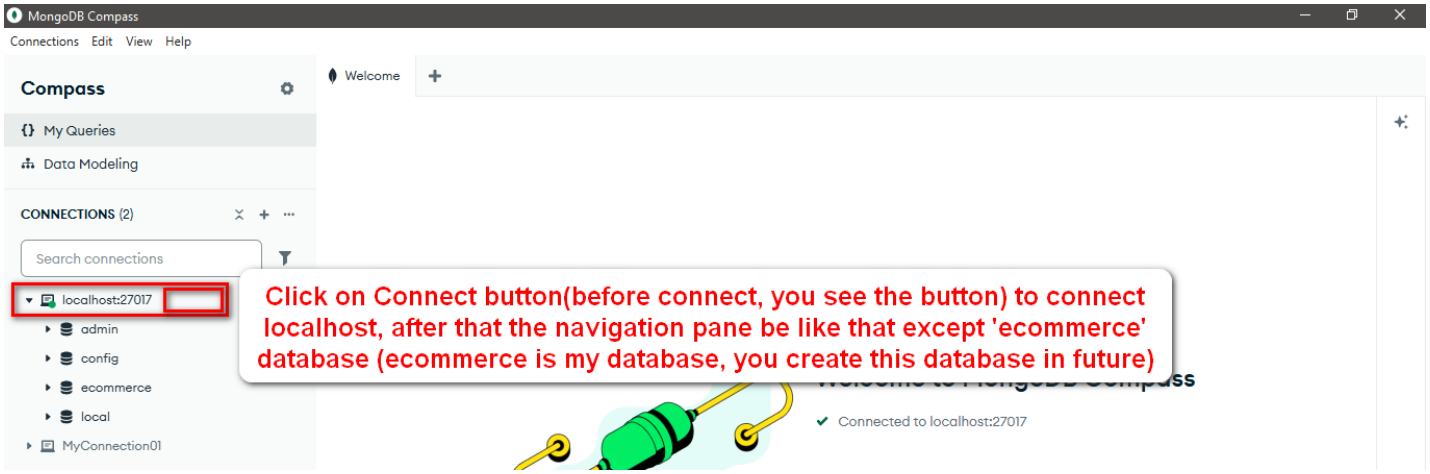
***Install MongoDB Community Server***

- **Download:** Go to mongodb.com → Community Server → download the installer for your OS.
  https://www.mongodb.com/try/download/community
  - **Run installer:** Choose "Complete" setup.
  - **Install MongoDB as a Service:** Checked by default; keep defaults (runs on port 27017).
  - **Tools:** Include MongoDB Shell if available.

**Verify MongoDB is running**

- **Check connection:** The default URI is mongodb://localhost:27017.
- **Optional shell:**
  - Run mongosh → then show dbs.
  - If it connects, your local MongoDB is running.

**Open your recently installed MongoDB on your system & also connect**



**Prepare project in VS Code**

- **Create folder:** ecommerce-realtime.
- **Open in VS Code:** File → Open Folder → ecommerce-realtime.
- **Files:** Create files exactly as named below and paste the code.

### Install Node.js and dependencies

- **Node.js:** Install LTS if not already.
- **Init and install:**
  - **Initialize:** npm init -y
  - **Install deps:** npm install express mongoose cors dotenv morgan socket.io
  - **Dev tool:** npm install --save-dev nodemon

### Run and test

- **Seed products:** npm run seed
- **Run dev:** npm run dev
- **Open browser:** http://localhost:3000
- **Admin page:** http://localhost:3000/admin.html
- **Realtime test:** Open two tabs; place an order in one; watch stock update in the other instantly.

### Folder structure

- **ecommerce-realtime/**
  - **package.json**
  - **.env**
  - **server.js**
  - **config/db.js**
  - **models/Product.js**
  - **models/Order.js**
  - **routes/api.js**
  - **routes/admin.js**
  - **seed/seed.js**
  - **public/**
    - **index.html**
    - **admin.html**
    - **css/styles.css**
    - **js/app.js**
    - **js/admin.js**
    - **img/** (optional)

## package.json

```
{
  "name": "ecommerce-realtime",
  "version": "1.2.0",
  "description": "Realtime e-Commerce using HTML5, CSS3, Bootstrap, NodeJS, jQuery, ExpressJS, MongoDB,
Socket.IO",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js",
    "seed": "node seed/seed.js",
    "zip": "zip -r ecommerce-realtime.zip . -x \"node_modules/*\""
  },
```

```
  "author": "Foysal",
  "license": "MIT",
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.0",
    "express": "^4.19.2",
    "mongoose": "^8.6.0",
    "morgan": "^1.10.0",
    "socket.io": "^4.7.5"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}
```

## .env

```
PORT=3000
MONGODB_URI=mongodb://localhost:27017
DB_NAME=ecommerce
```

## server.js

```
require('dotenv').config();
const path = require('path');
const express = require('express');
const morgan = require('morgan');
const cors = require('cors');
const http = require('http');
const { Server } = require('socket.io');

const connectDB = require('./config/db');
const apiRoutes = require('./routes/api');
const adminRoutes = require('./routes/admin');

const app = express();
const server = http.createServer(app);
const io = new Server(server, {
  cors: { origin: '*' }
});

// Middleware
app.use(morgan('dev'));
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Static files
app.use(express.static(path.join(__dirname, 'public')));

// API routes
app.use('/api', apiRoutes);
app.use('/admin', adminRoutes);
```

```
// Socket.IO
io.on('connection', (socket) => {
  console.log('Client connected:', socket.id);
  socket.on('disconnect', () => console.log('Client disconnected:', socket.id));
});

// Make io available in routes
app.set('io', io);

const PORT = process.env.PORT || 3000;

connectDB()
  .then(() => {
    server.listen(PORT, () => {
      console.log(`Server running at http://localhost:${PORT}`);
    });
  })
  .catch((err) => {
    console.error('Failed to connect to MongoDB:', err.message);
    process.exit(1);
  });
```

## config/db.js

```
const mongoose = require('mongoose');

async function connectDB() {
  const uri = process.env.MONGODB_URI;
  const dbName = process.env.DB_NAME || 'ecommerce';

  if (!uri) {
    throw new Error('MONGODB_URI is missing in .env');
  }

  await mongoose.connect(uri, { dbName });
  console.log(`MongoDB connected to database: ${dbName}`);
}

module.exports = connectDB;
```

## models/Product.js

```
const mongoose = require('mongoose');

const productSchema = new mongoose.Schema(
  {
    title: { type: String, required: true, trim: true },
    description: { type: String, default: '' },
    price: { type: Number, required: true, min: 0 },
    currency: { type: String, default: 'BDT' },
    imageUrl: { type: String, default: '' },
    stock: { type: Number, default: 0, min: 0 },
    category: { type: String, default: 'General' }
  },
```

```
  { timestamps: true }
);

module.exports = mongoose.model('Product', productSchema);
```

## models/Order.js

```
const mongoose = require('mongoose');

const orderItemSchema = new mongoose.Schema(
  {
    productId: { type: mongoose.Schema.Types.ObjectId, ref: 'Product', required: true },
    title: { type: String, required: true },
    quantity: { type: Number, required: true, min: 1 },
    price: { type: Number, required: true, min: 0 }
  },
  { _id: false }
);

const orderSchema = new mongoose.Schema(
  {
    customerName: { type: String, required: true },
    customerEmail: { type: String, required: true },
    items: { type: [orderItemSchema], required: true },
    totalAmount: { type: Number, required: true, min: 0 },
    status: { type: String, default: 'PLACED', enum: ['PLACED', 'CONFIRMED', 'SHIPPED', 'DELIVERED', 'CANCELLED'] }
  },
  { timestamps: true }
);

module.exports = mongoose.model('Order', orderSchema);
```

## routes/api.js

```
const express = require('express');
const Product = require('../models/Product');
const Order = require('../models/Order');

const router = express.Router();

// Health check
router.get('/health', (req, res) => res.json({ ok: true, timestamp: new Date().toISOString() }));

// Products list
router.get('/products', async (req, res) => {
  try {
    const products = await Product.find().sort({ createdAt: -1 });
    res.json(products);
  } catch {
    res.status(500).json({ message: 'Failed to fetch products' });
  }
});

// Create order
```

```
router.post('/orders', async (req, res) => {
  try {
    const { customerName, customerEmail, items } = req.body;
    if (!customerName || !customerEmail || !Array.isArray(items) || items.length === 0) {
      return res.status(400).json({ message: 'Invalid order payload' });
    }

    let totalAmount = 0;
    for (const item of items) {
      const product = await Product.findById(item.productId);
      if (!product) return res.status(400).json({ message: 'Invalid product in order' });
      if (product.stock < item.quantity) {
        return res.status(400).json({ message: `Insufficient stock for ${product.title}` });
      }
      totalAmount += product.price * item.quantity;
      product.stock -= item.quantity;
      await product.save();
      item.title = product.title;
      item.price = product.price;
    }

    const order = await Order.create({
      customerName,
      customerEmail,
      items,
      totalAmount
    });

    // Emit realtime events
    const io = req.app.get('io');
    io.emit('productUpdate'); // products changed → refresh UI
    io.emit('newOrder', order); // a new order appeared

    res.status(201).json({ message: 'Order placed', order });
  } catch (err) {
    console.error(err);
    res.status(500).json({ message: 'Failed to place order' });
  }
});

// Orders list
router.get('/orders', async (req, res) => {
  try {
    const orders = await Order.find().sort({ createdAt: -1 });
    res.json(orders);
  } catch {
    res.status(500).json({ message: 'Failed to fetch orders' });
  }
});

module.exports = router;
```

## routes/admin.js

```
const express = require('express');
const Product = require('../models/Product');

const router = express.Router();

// Create product
router.post('/products', async (req, res) => {
  try {
    const product = await Product.create(req.body);
    const io = req.app.get('io');
    io.emit('productUpdate');
    res.status(201).json(product);
  } catch (err) {
    res.status(500).json({ message: 'Failed to create product' });
  }
});

// Read all products
router.get('/products', async (req, res) => {
  try {
    const products = await Product.find().sort({ createdAt: -1 });
    res.json(products);
  } catch {
    res.status(500).json({ message: 'Failed to fetch products' });
  }
});

// Update product
router.put('/products/:id', async (req, res) => {
  try {
    const product = await Product.findByIdAndUpdate(req.params.id, req.body, { new: true });
    if (!product) return res.status(404).json({ message: 'Product not found' });
    const io = req.app.get('io');
    io.emit('productUpdate');
    res.json(product);
  } catch {
    res.status(500).json({ message: 'Failed to update product' });
  }
});

// Delete product
router.delete('/products/:id', async (req, res) => {
  try {
    const product = await Product.findByIdAndDelete(req.params.id);
    if (!product) return res.status(404).json({ message: 'Product not found' });
    const io = req.app.get('io');
    io.emit('productUpdate');
    res.json({ message: 'Product deleted' });
  } catch {
    res.status(500).json({ message: 'Failed to delete product' });
  }
});

module.exports = router;
```

## seed/seed.js

```
require('dotenv').config();
const mongoose = require('mongoose');
const Product = require('../models/Product');

const uri = process.env.MONGODB_URI;
const dbName = process.env.DB_NAME || 'ecommerce';

const sampleProducts = [
  {
    title: 'NVIT Hoodie',
    description: 'Comfortable cotton hoodie with NVIT branding',
    price: 1499,
    currency: 'BDT',
    imageUrl: 'https://via.placeholder.com/600x400?text=NVIT+Hoodie',
    stock: 20,
    category: 'Apparel'
  },
  {
    title: 'Power BI Guidebook',
    description: 'Practical guide for dashboards and analytics',
    price: 999,
    currency: 'BDT',
    imageUrl: 'https://via.placeholder.com/600x400?text=Power+BI+Guidebook',
    stock: 50,
    category: 'Books'
  },
  {
    title: 'Excel Shortcuts Keyboard',
    description: 'Printed shortcuts for faster data work',
    price: 499,
    currency: 'BDT',
    imageUrl: 'https://via.placeholder.com/600x400?text=Excel+Keyboard',
    stock: 35,
    category: 'Accessories'
  }
];

(async () => {
  try {
    await mongoose.connect(uri, { dbName });
    console.log('Connected. Seeding products...');
    await Product.deleteMany({});
    await Product.insertMany(sampleProducts);
    console.log('Seeding complete.');
    process.exit(0);
  } catch (err) {
    console.error('Seeding failed:', err.message);
    process.exit(1);
  }
})();
```

## public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta http-equiv="X-UA-Compatible" content="IE=edge" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Realtime e-Commerce</title>

  <!-- Bootstrap CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet" />
  <link rel="stylesheet" href="/css/styles.css" />
</head>
<body>
  <!-- Navbar -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container">
      <a class="navbar-brand fw-bold" href="#">NVIT Store</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navContent">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div id="navContent" class="collapse navbar-collapse">
        <ul class="navbar-nav ms-auto">
          <li class="nav-item"><a class="nav-link active" href="#">Home</a></li>
          <li class="nav-item"><a class="nav-link" href="#" id="viewOrdersLink">Orders</a></li>
          <li class="nav-item"><a class="nav-link" href="#" id="viewCartLink">Cart (<span
id="cartCount">0</span>)</a></li>
          <li class="nav-item"><a class="nav-link" href="/admin.html">Admin</a></li>
        </ul>
      </div>
    </div>
  </nav>

  <!-- Hero -->
  <header class="bg-primary text-white py-5">
    <div class="container text-center">
      <h1 class="display-5">Realtime e-Commerce</h1>
      <p class="lead">HTML5, CSS3, Bootstrap, jQuery, Node.js, Express.js, MongoDB (Local), Socket.IO</p>
    </div>
  </header>

  <!-- Products -->
  <section class="container my-4">
    <h2 class="mb-3">Products</h2>
    <div id="productGrid" class="row g-4"></div>
  </section>

  <!-- Cart Modal -->
  <div class="modal fade" id="cartModal" tabindex="-1" aria-hidden="true">
    <div class="modal-dialog modal-lg">
      <div class="modal-content">
        <div class="modal-header">
          <h5 class="modal-title">Your Cart</h5>
```

```
        <button type="button" class="btn-close" data-bs-dismiss="modal"></button>
      </div>
      <div class="modal-body">
        <div id="cartItems"></div>
        <hr />
        <form id="checkoutForm" class="row g-3">
          <div class="col-md-6">
            <label class="form-label">Name</label>
            <input type="text" class="form-control" id="customerName" required />
          </div>
          <div class="col-md-6">
            <label class="form-label">Email</label>
            <input type="email" class="form-control" id="customerEmail" required />
          </div>
          <div class="col-12 d-flex justify-content-between align-items-center">
            <h5>Total: <span id="cartTotal">0</span> BDT</h5>
            <button type="submit" class="btn btn-success">Place Order</button>
          </div>
        </form>
        <div id="checkoutMsg" class="mt-3"></div>
      </div>
    </div>
  </div>
</div>

<!-- Orders Modal -->
<div class="modal fade" id="ordersModal" tabindex="-1" aria-hidden="true">
  <div class="modal-dialog modal-lg">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title">Recent Orders</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal"></button>
      </div>
      <div class="modal-body">
        <div id="ordersList"></div>
      </div>
    </div>
  </div>
</div>

<!-- jQuery + Bootstrap JS -->
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
<!-- Socket.IO client -->
<script src="/socket.io/socket.io.js"></script>
<!-- App JS -->
<script src="/js/app.js"></script>
</body>
</html>
```

## public/admin.html

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
  <meta charset="UTF-8" />
  <title>Admin Dashboard</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet" />
</head>
<body class="bg-light">
  <div class="container py-4">
    <h1 class="mb-4">Admin Dashboard</h1>
    <p class="mb-3"><a href="/" class="btn btn-outline-secondary btn-sm">← Back to Store</a></p>

    <!-- Add Product Form -->
    <form id="addProductForm" class="row g-3 mb-4">
      <div class="col-md-4"><input type="text" class="form-control" id="title" placeholder="Title" required /></div>
      <div class="col-md-4"><input type="number" class="form-control" id="price" placeholder="Price" required
/></div>
      <div class="col-md-4"><input type="number" class="form-control" id="stock" placeholder="Stock" required
/></div>
      <div class="col-md-6"><input type="text" class="form-control" id="imageUrl" placeholder="Image URL" /></div>
      <div class="col-md-6"><input type="text" class="form-control" id="category" placeholder="Category" /></div>
      <div class="col-md-12"><textarea class="form-control" id="description"
placeholder="Description"></textarea></div>
      <div class="col-md-12"><button type="submit" class="btn btn-primary">Add Product</button></div>
    </form>

    <!-- Product List -->
    <table class="table table-bordered">
      <thead><tr><th>Title</th><th>Price</th><th>Stock</th><th>Actions</th></tr></thead>
      <tbody id="productTable"></tbody>
    </table>
  </div>

  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
  <script src="/socket.io/socket.io.js"></script>
  <script src="/js/admin.js"></script>
</body>
</html>
```

## public/css/styles.css

```
body {
  background-color: #f8f9fa;
}

.card-title {
  font-weight: 600;
}

.price-tag {
  font-size: 1rem;
  font-weight: 600;
  color: #0d6efd;
}

.navbar-brand {
```

```
  letter-spacing: 0.5px;
}

#cartItems .cart-row {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 6px 0;
}

#checkoutMsg {
  min-height: 24px;
}
```

## public/js/app.js

```
// Frontend logic with jQuery + Socket.IO realtime

const cartKey = 'nvit_cart';
const socket = io();

function getCart() {
  const raw = localStorage.getItem(cartKey);
  return raw ? JSON.parse(raw) : [];
}

function saveCart(items) {
  localStorage.setItem(cartKey, JSON.stringify(items));
  updateCartCount();
}

function updateCartCount() {
  const items = getCart();
  const totalQty = items.reduce((sum, it) => sum + it.quantity, 0);
  $('#cartCount').text(totalQty);
}

function addToCart(product, qty = 1) {
  const cart = getCart();
  const existing = cart.find((c) => c.productId === product._id);
  if (existing) {
    existing.quantity += qty;
  } else {
    cart.push({
      productId: product._id,
      title: product.title,
      price: product.price,
      quantity: qty
    });
  }
  saveCart(cart);
}

function renderProducts(products) {
```

```
  const grid = $('#productGrid');
  grid.empty();

  products.forEach((p) => {
    const col = $(`
      <div class="col-md-4 col-sm-6">
        <div class="card h-100">
          <img src="${p.imageUrl || 'https://via.placeholder.com/600x400?text=Product'}" class="card-img-top"
alt="${p.title}" />
          <div class="card-body d-flex flex-column">
            <h5 class="card-title">${p.title}</h5>
            <p class="card-text">${p.description || ''}</p>
            <div class="d-flex justify-content-between align-items-center mt-auto">
              <span class="price-tag">${p.price} ${p.currency || 'BDT'}</span>
              <button class="btn btn-primary btn-sm" ${p.stock <= 0 ? 'disabled' : ''}>${p.stock <= 0 ? 'Out of Stock' : 'Add
to Cart'}</button>
            </div>
            <small class="text-muted mt-1">Stock: ${p.stock}</small>
          </div>
        </div>
      </div>
    `);

    col.find('button').on('click', () => addToCart(p, 1));
    grid.append(col);
  });
}

function renderCart() {
  const items = getCart();
  const container = $('#cartItems');
  container.empty();

  if (items.length === 0) {
    container.html('<p class="text-muted">Your cart is empty.</p>');
    $('#cartTotal').text(0);
    return;
  }

  let total = 0;
  items.forEach((it, idx) => {
    total += it.price * it.quantity;
    const row = $(`
      <div class="cart-row">
        <div><strong>${it.title}</strong> — ${it.price} BDT x ${it.quantity}</div>
        <div>
          <button class="btn btn-outline-secondary btn-sm me-2" data-idx="${idx}" data-action="dec">-</button>
          <button class="btn btn-outline-secondary btn-sm me-2" data-idx="${idx}" data-action="inc">+</button>
          <button class="btn btn-outline-danger btn-sm" data-idx="${idx}" data-action="remove">Remove</button>
        </div>
      </div>
    `);
    container.append(row);
  });
```

```
  $('#cartTotal').text(total);

  container.find('button').on('click', function () {
    const idx = parseInt($(this).data('idx'), 10);
    const action = $(this).data('action');
    const cart = getCart();

    if (action === 'dec') {
      cart[idx].quantity = Math.max(1, cart[idx].quantity - 1);
    } else if (action === 'inc') {
      cart[idx].quantity += 1;
    } else if (action === 'remove') {
      cart.splice(idx, 1);
    }

    saveCart(cart);
    renderCart();
  });
}

function placeOrder(name, email) {
  const items = getCart();
  if (items.length === 0) return { ok: false, message: 'Cart is empty' };

  const payload = {
    customerName: name,
    customerEmail: email,
    items: items.map((it) => ({
      productId: it.productId,
      quantity: it.quantity
    }))
  };

  return $.ajax({
    url: '/api/orders',
    method: 'POST',
    contentType: 'application/json',
    data: JSON.stringify(payload)
  });
}

function fetchOrders() {
  return $.get('/api/orders');
}

function loadProducts() {
  $.get('/api/products').then(renderProducts);
}

$(function () {
  updateCartCount();
  loadProducts();
```

```
// Realtime events
socket.on('productUpdate', () => {
  loadProducts();
});

socket.on('newOrder', (order) => {
  console.log('New order:', order);
  if ($('#ordersModal').hasClass('show')) {
    fetchOrders().then((orders) => {
      const container = $('#ordersList');
      container.empty();
      orders.forEach((o) => {
        const itemsText = o.items.map((i) => `${i.title} x ${i.quantity} (${i.price} BDT)`).join(', ');
        const card = $(`
          <div class="border rounded p-2 mb-2">
            <div><strong>${o.customerName}</strong> — ${o.customerEmail}</div>
            <div>Items: ${itemsText}</div>
            <div>Total: ${o.totalAmount} BDT</div>
            <small class="text-muted">Status: ${o.status} | ${new Date(o.createdAt).toLocaleString()}</small>
          </div>
        `);
        container.append(card);
      });
    });
  }
});

// Cart modal
const cartModal = new bootstrap.Modal('#cartModal');
$('#viewCartLink').on('click', (e) => {
  e.preventDefault();
  renderCart();
  cartModal.show();
});

// Orders modal
const ordersModal = new bootstrap.Modal('#ordersModal');
$('#viewOrdersLink').on('click', (e) => {
  e.preventDefault();
  fetchOrders().then((orders) => {
    const container = $('#ordersList');
    container.empty();
    if (!orders || orders.length === 0) {
      container.html('<p class="text-muted">No orders yet.</p>');
      return;
    }
    orders.forEach((o) => {
      const itemsText = o.items.map((i) => `${i.title} x ${i.quantity} (${i.price} BDT)`).join(', ');
      const card = $(`
        <div class="border rounded p-2 mb-2">
          <div><strong>${o.customerName}</strong> — ${o.customerEmail}</div>
          <div>Items: ${itemsText}</div>
          <div>Total: ${o.totalAmount} BDT</div>
          <small class="text-muted">Status: ${o.status} | ${new Date(o.createdAt).toLocaleString()}</small>
```

```
        </div>
      `);
      container.append(card);
    });
    ordersModal.show();
  });
});

// Checkout
$('#checkoutForm').on('submit', function (e) {
  e.preventDefault();
  const name = $('#customerName').val().trim();
  const email = $('#customerEmail').val().trim();

  $('#checkoutMsg').removeClass().addClass('text-muted').text('Placing order...');
  placeOrder(name, email)
    .then(() => {
      $('#checkoutMsg').removeClass().addClass('text-success').text('Order placed successfully!');
      saveCart([]); // clear cart
      renderCart();
    })
    .catch((xhr) => {
      const msg = xhr?.responseJSON?.message || 'Order failed';
      $('#checkoutMsg').removeClass().addClass('text-danger').text(msg);
    });
});
});
```

## public/js/admin.js

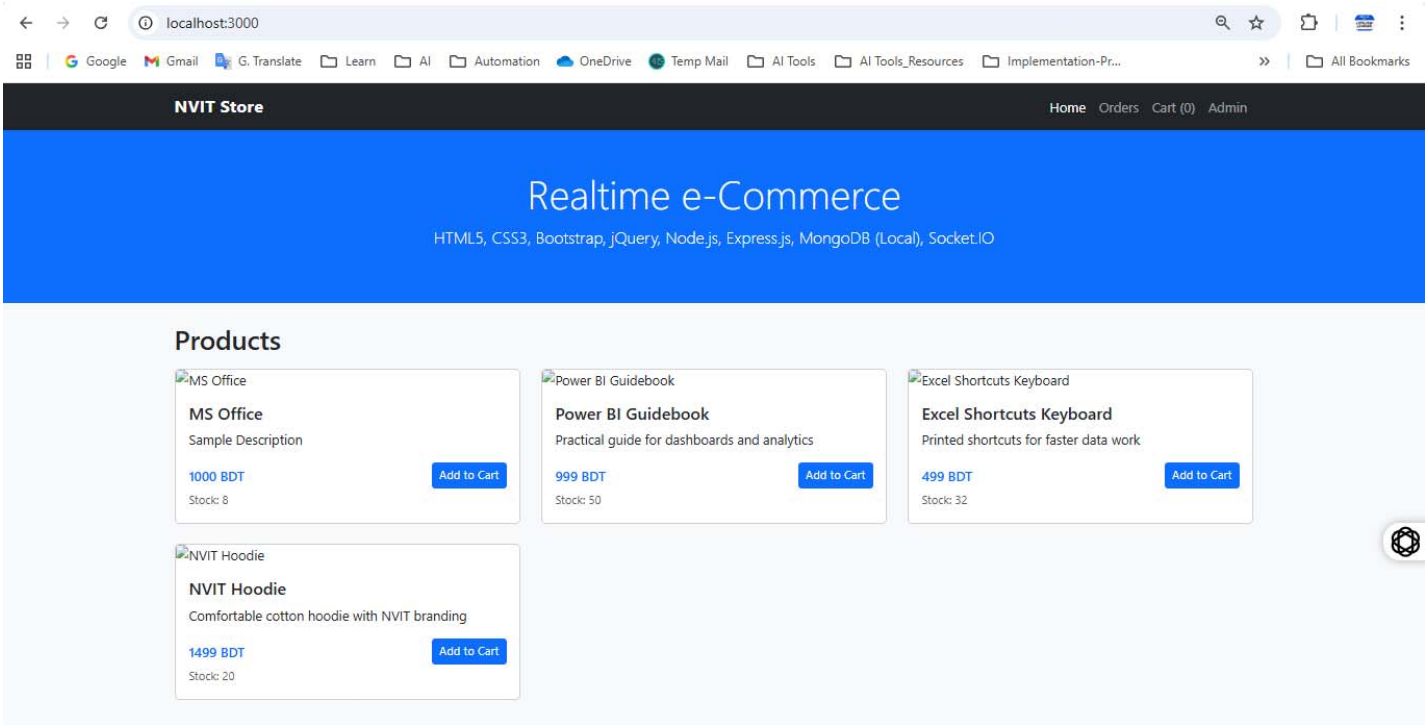
```
const socket = io();

function loadProducts() {
  $.get('/admin/products').then((products) => {
    const table = $('#productTable');
    table.empty();
    products.forEach((p) => {
      const row = $(`
        <tr>
          <td>${p.title}</td>
          <td><input type="number" class="form-control form-control-sm price-input" data-id="${p._id}"
value="${p.price}" /></td>
          <td><input type="number" class="form-control form-control-sm stock-input" data-id="${p._id}"
value="${p.stock}" /></td>
          <td>
            <button class="btn btn-success btn-sm update-btn" data-id="${p._id}">Update</button>
            <button class="btn btn-danger btn-sm delete-btn" data-id="${p._id}">Delete</button>
          </td>
        </tr>
      `);
      table.append(row);
    });
  });
}
```

```
$(function () {
  loadProducts();

  // Realtime refresh
  socket.on('productUpdate', () => {
    loadProducts();
  });

  // Add product
  $('#addProductForm').on('submit', function (e) {
    e.preventDefault();
    const payload = {
      title: $('#title').val(),
      price: Number($('#price').val()),
      stock: Number($('#stock').val()),
      imageUrl: $('#imageUrl').val(),
      category: $('#category').val(),
      description: $('#description').val()
    };
    $.post('/admin/products', payload).then(() => {
      this.reset();
      loadProducts();
    });
  });

  // Update product
  $(document).on('click', '.update-btn', function () {
    const id = $(this).data('id');
    const price = Number($(`.price-input[data-id="${id}"]`).val());
    const stock = Number($(`.stock-input[data-id="${id}"]`).val());
    $.ajax({
      url: `/admin/products/${id}`,
      method: 'PUT',
      contentType: 'application/json',
      data: JSON.stringify({ price, stock })
    }).then(() => loadProducts());
  });

  // Delete product
  $(document).on('click', '.delete-btn', function () {
    const id = $(this).data('id');
    $.ajax({ url: `/admin/products/${id}`, method: 'DELETE' }).then(() => loadProducts());
  });
});
```

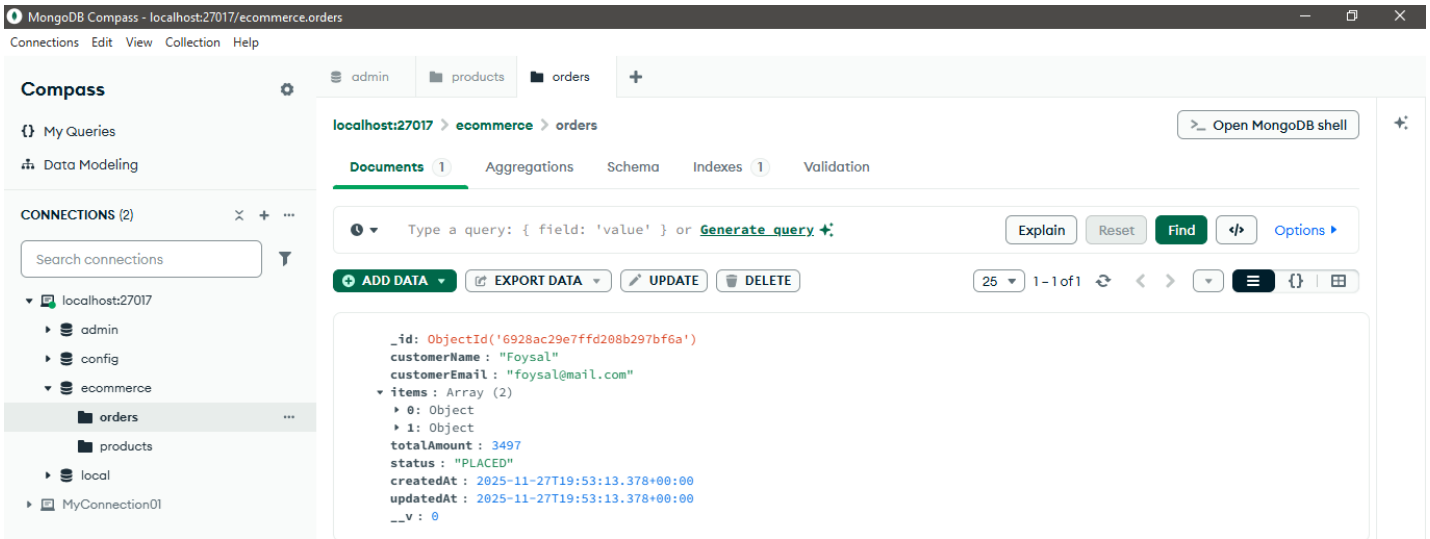
## Run and test

- **Run dev:** npm run dev
- **Open browser:** http://localhost:3000
- **Admin page:** http://localhost:3000/admin.html
- **Realtime test:** Open two tabs; place an order in one; watch stock update in the other instantly.

**Now, you can place an order through Cart & also you can add a product from admin section. All tasks perform real-time.**




## Check the database table values




## Project Structure